

Network Programming 2026 - Assignment #4

Course: Network Programming

Assignment: Assignment #4 (Chapters 9, 10, 11)

Name: 홍성주

Student ID: 20221528

Question 1. HTTP Dictionary Lookup

1. Screenshots



```
Terminal Local x + v
PS C:\Users\akftj\Desktop\Network Programming 2026_Assignment 4_templates\templates> python solution1_server.py
HTTP Dictionary Server listening on 127.0.0.1:8080
[GET] /define?keyword=HTTP -> 200 OK
[GET] /define?keyword=Host -> 200 OK
[GET] /define?keyword=StatusCode -> 200 OK
[GET] /define?keyword=xyz -> 404 Not Found

PS C:\Users\akftj\Desktop\Network Programming 2026_Assignment 4_templates\templates> python solution1_client.py
HTTP Dictionary Client - server at 127.0.0.1:8080
Type a keyword to look up, or 'quit' to exit.

> HTTP
Definition: HyperText Transfer Protocol - the foundation of data communication on the World Wide Web.
> Host
Definition: The HTTP Host header specifies the domain name of the server to which the request is being sent.
> StatusCode
Definition: A three-digit number in an HTTP response indicating the result of the request (e.g. 200 OK or 404 Not Found).
> xyz
Error 404: Keyword not found.
> quit
Disconnected.
PS C:\Users\akftj\Desktop\Network Programming 2026_Assignment 4_templates\templates>
```

2. My Code Logic

For this problem, I built a raw HTTP/1.1 server and client using only the socket and csv modules, exactly as required.

On the server side, I load the keyword-definition pairs from keywords.csv into a Python dictionary at startup, using the keyword as the key in lowercase. That way I can do a case-insensitive lookup in $O(1)$ time for every request. I set `SO_REUSEADDR` on the listening socket so I can restart the server immediately after stopping it without getting an "Address already in use" error.

For each connection, I accumulate bytes in a loop until I see `␣␣␣␣␣␣` — the blank line that signals the end of the HTTP headers. Then I parse only the first line (the request line) to extract the method and the path. The keyword is pulled out of the query string by splitting on `?` and scanning the key-value pairs for one that starts with `keyword=`. I also wrote a small `url_decode()` function using `re.sub()` so that percent-encoded characters (like `%20` for a space) are handled correctly without importing `urllib`.

The response is built manually as a byte string: a status line (HTTP/1.1 200 OK or HTTP/1.1 404 Not Found), followed by Content-Type, Content-Length, and Connection: close headers, then a blank line, then the body. Content-Length is important because it tells the client exactly how many bytes to read as the body, so Connection: close is enough for the client to know when to stop receiving.

On the client side, I open a fresh socket for every lookup because the server sends Connection: close, which means the server closes the connection after each response and the same socket cannot be reused. After calling `sendall()` with the GET request, I read in a loop until the server closes the connection, then split the response at `␣␣␣␣␣␣` and display only the body.

3. How I Handled Errors

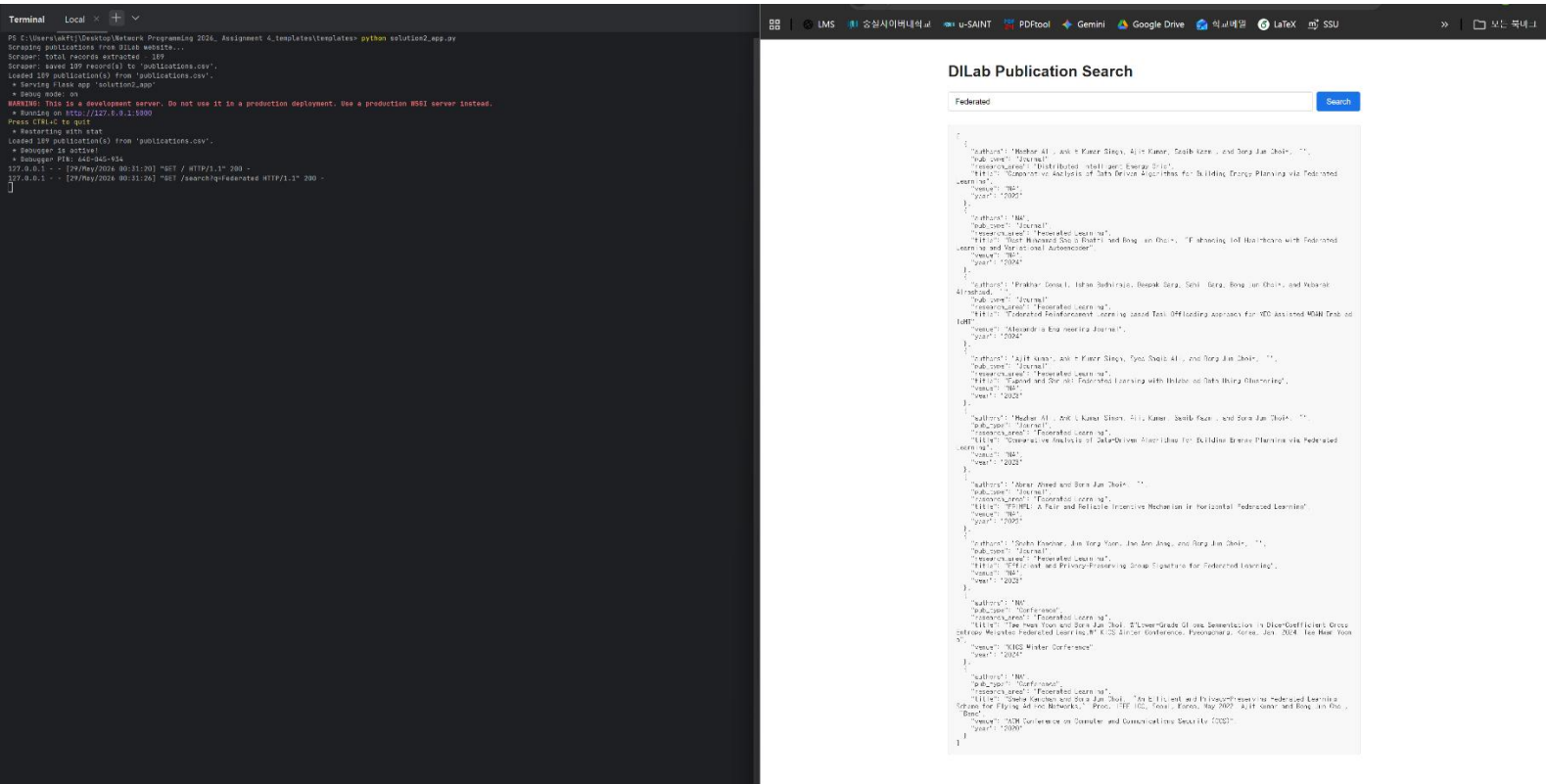
On the server side, I wrapped the entire request-handling logic in a broad try/except Exception block so that a malformed request from one client cannot crash the whole server. More specific cases — a missing keyword parameter or an unsupported HTTP

method — are caught early and return the appropriate 400 or 405 response instead of letting the exception propagate. The client socket is always closed in a finally block regardless of what happens inside handle_request().

On the client side, I separately catch ConnectionRefusedError (server is not running) and OSError (general socket failure) so the user sees a clear, actionable message instead of a raw Python traceback. EOFError and KeyboardInterrupt are caught at the input prompt so the client exits cleanly when the user presses Ctrl+C or Ctrl+D.

Question 2. Flask Web Application with Web Scraping

1. Screenshots




```
Terminal Local + -
PS C:\Users\labf1\Desktop\Network Programming 2026, Assignment 4_templates\templates> python solution2_app.py
Scraping publications from DLab website...
Scraper: total records extracted - 199
Scraper: saved 199 records to 'publications.csv'.
Loaded 199 publication(s) from 'publications.csv'.
+ Saving Flask app 'solution2_app'
+ Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
+ Restarting with stat
loader: 199 publication(s) from 'publications.csv'.
+ Debugger is active!
+ Debugger PID: 640-645-654
127.0.0.1 - [29/May/2026 00:31:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - [29/May/2026 00:31:26] "GET /search/federated HTTP/1.1" 200
127.0.0.1 - [29/May/2026 00:31:52] "GET /search/federated HTTP/1.1" 200
127.0.0.1 - [29/May/2026 00:32:16] "GET /search/song-jun+choi HTTP/1.1" 200
]

[93]
authors      : Since S. Woo, Kyoung Joo Jung, and Bong Jun Choi, "
pub_type     : Conference
research_area : Federated Learning
title        : Survey on Current Password Composition Policies
venue        : Korean Institute of Information Security and Cryptology Conference (KIISS)
year         : 2018

[94]
authors      : Pratik Musale, Dain Bask, and Bong Jun Choi, "
pub_type     : Conference
research_area : Federated Learning
title        : Lightweight Bell-Based Authentication Technique for IoT using Subconscious Level Activities
venue        : IEEE World Forum on Internet of Things (WF-IoT)
year         : 2018

[95]
authors      : Mostash Saxeena, Santiago Grijalva, and Bong Jun Choi, "
pub_type     : Conference
research_area : Federated Learning
title        : Securing Restricted Publisher-Subscriber Communications in Smart Grid Substations
venue        : NA
year         : 2018

[96]
authors      : NA
pub_type     : Conference
research_area : Federated Learning
title        : Pratik Musale, Dain Bask, Since Woo, and Bong Jun Choi, "Lightweight Authentication for IoT," Proc. ACM CoNEXT, Seoul, South Korea, Dec. 2017. (Student Workshop)
authors      : Mostash Saxeena, Bong Jun Choi, and San
venue        : IEEE ICC
year         : 2017

[97]
authors      : NA
pub_type     : Conference
research_area : Federated Learning
title        : Mostash Saxeena and Bong Jun Choi, "A Framework for Evaluating Hybrid Intrusion Detection System and Change in Electricity Prices in the API Network," Proc. International Smart Grid Conference (ISGC),
venue        : IEEE TrustCom
year         : 2018

[98]
authors      : Shinyoung Che, Hongwei Li, and Bong Jun Choi, "
pub_type     : Conference
research_area : Federated Learning
title        : PALDA: Efficient Privacy-preserving Authentication for Lossless Data Aggregation in Smart Grids
venue        : IEEE SmartGridComm
year         : 2014

[99]
authors      : Qi Li, Hongwei Li, Yi Yang, Hongwei Luo, Bong Jun Choi, and Liang Zhou, "
pub_type     : Conference
research_area : Federated Learning
title        : ESO: An Efficient and Secure Outsourcing Scheme for Smart Grid
venue        : International Conference on Wireless Communications and Signal Processing
year         : 2013

[100]
authors      : NA
pub_type     : Conference
research_area : Federated Learning
title        : Books (Book Chapters) Ajit Kumar, Bong Jun Choi, K S Kuppaswamy, and S Aghila, "Malware attacks: Dimensions, Impact, and Defences," Book chapter in Advances in Network Inspired Cyber-Security and Resili
venue        : NA
year         : 2013

[101]
authors      : NA
pub_type     : Conference
research_area : Data Analytics
title        : Bone Bui, Vu, Jaya Thomas, Shinyoung Che, Pradip Das, and Bong Jun Choi, "Next Generation Business Intelligence and Analytics," Proc. International Conference on Business and Information Management (CI
venue        : International Conference on Business and Information Management (ICBIM)
year         : 2018

> QUIT
Exiting.
PS C:\Users\labf1\Desktop\Network Programming 2026, Assignment 4_templates\templates>
```

2. My Code Logic

This question has three parts: a web scraper, a Flask application, and a client script.

Web Scraper. The scraper runs at startup, before the Flask server begins accepting requests. To prevent the scraper from running twice when Flask's Werkzeug reloader forks a child process, I check the WERKZEUG_RUN_MAIN environment variable — if it is not 'true', we are in the parent process and should scrape; the child inherits the already-saved CSV and just loads it.

I used requests to fetch the DILab publications page with a realistic User-Agent header to avoid being blocked. Then I passed the HTML to BeautifulSoup for parsing. The scraper walks all text-bearing elements in document order and looks for research area headings (detected by matching keywords like "internet of things" or "federated

learning") and sub-headings ("Journals", "Conferences"). Text that falls under a known sub-heading is accumulated in a buffer and flushed into a record when the next heading is detected.

Since Google Sites pages are often rendered client-side by JavaScript and may not expose all content in the static HTML, I added a second strategy: scanning embedded `<script>` tags with a regex that looks for long strings containing academic venue keywords (IEEE, ACM, Journal, etc.). This fallback ensures the scraper returns something useful even when the HTML traversal finds nothing.

The extracted records are saved to `publications.csv` with the columns `research_area`, `pub_type`, `title`, `authors`, `venue`, `year`. Any field that could not be extracted is stored as `"NA"`.

Flask Application. The root route (GET `/`) serves a hand-written HTML page with a styled text input and a Search button. A small inline JavaScript function fetches `/search?q=<term>` with the Fetch API and displays the raw JSON response in a `<pre>` block, so the user can search without a page reload.

The search route (GET `/search`) reads the `q` parameter, lowercases it, and scans the in-memory publications list for records whose title or authors field contains the query as a substring. The matches are returned as JSON using Flask's `jsonify()`. If nothing matches, a

message dict is returned with HTTP 200 instead of an error code, which matches the expected API behaviour from the spec.

Client Script. The client uses `requests.get()` to call `/search`, passing the user's term as the `q` parameter. When the result is a list I print each publication in a numbered block with aligned field names; when it is a dict (a message or error), I print it directly.

3. How I Handled Errors

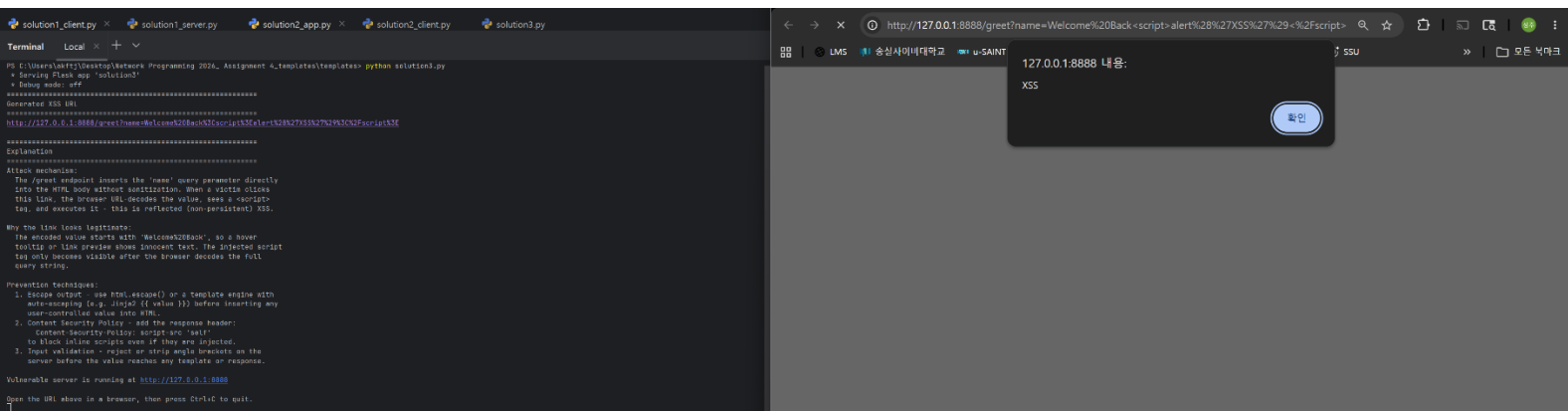
In the scraper, each network failure type is caught separately: `ConnectionError`, `Timeout`, `HTTPError`, and the base `RequestException`. This way the server does not crash even if the DILab website is unreachable. If the scraper returns zero records, the app falls back to loading any existing `publications.csv` from a previous run.

File I/O errors when writing or reading the CSV are caught with `OSError` so the app prints a clear message instead of crashing. The `load_csv()` function catches `FileNotFoundError` separately and starts with an empty list, which means the search route will return "no results" rather than a 500 error.

In the client, I catch `ConnectionError` (app not running), `Timeout`, `HTTPError`, and the base `RequestException` so the user always sees a readable message. `EOFError` and `KeyboardInterrupt` from the input prompt exit the loop cleanly.

Question 3. Cross-Site Scripting (XSS) URL Generator

1. Screenshot



2. My Code Logic

The script starts a minimal Flask server in a daemon thread. The server exposes a single route (/greet) that reads the name query parameter and inserts it directly into an HTML response without any escaping — this is the deliberate vulnerability.

The build_xss_url() function constructs the attack URL. It concatenates an innocent-looking string ("Welcome Back") with the script payload (<script>alert('XSS')</script>) and percent-encodes the combined string using urllib.parse.quote(). The result is a URL like:

http://127.0.0.1:8888/greet?name=Welcome%20Back%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E

When a victim visits this URL, the browser decodes the query parameter and the server embeds the raw value into the HTML response body. The browser then parses the HTML

and encounters the `<script>` tag, executing `alert('XSS')`. The "Welcome Back" text before the tag makes the link look harmless in a hover preview or shortened URL.

I used a daemon thread for the Flask server so the main thread can keep running to print output and wait for Ctrl+C. The `time.sleep(1)` after starting the thread gives Flask enough time to bind the port before the URL is printed.

3. How I Handled Errors

The script wraps the Flask import in a try/except so that it still runs and prints the explanation even if Flask is not installed (the server just does not start). The `_FLASK_AVAILABLE` flag controls whether the server thread is launched at all.

The main loop uses a try/except KeyboardInterrupt around the `while True: time.sleep(1)` loop so the user can press Ctrl+C to quit cleanly without a traceback. Werkzeug's request log is silenced by setting its logger level to ERROR, keeping the terminal output focused on the generated URL and explanation.